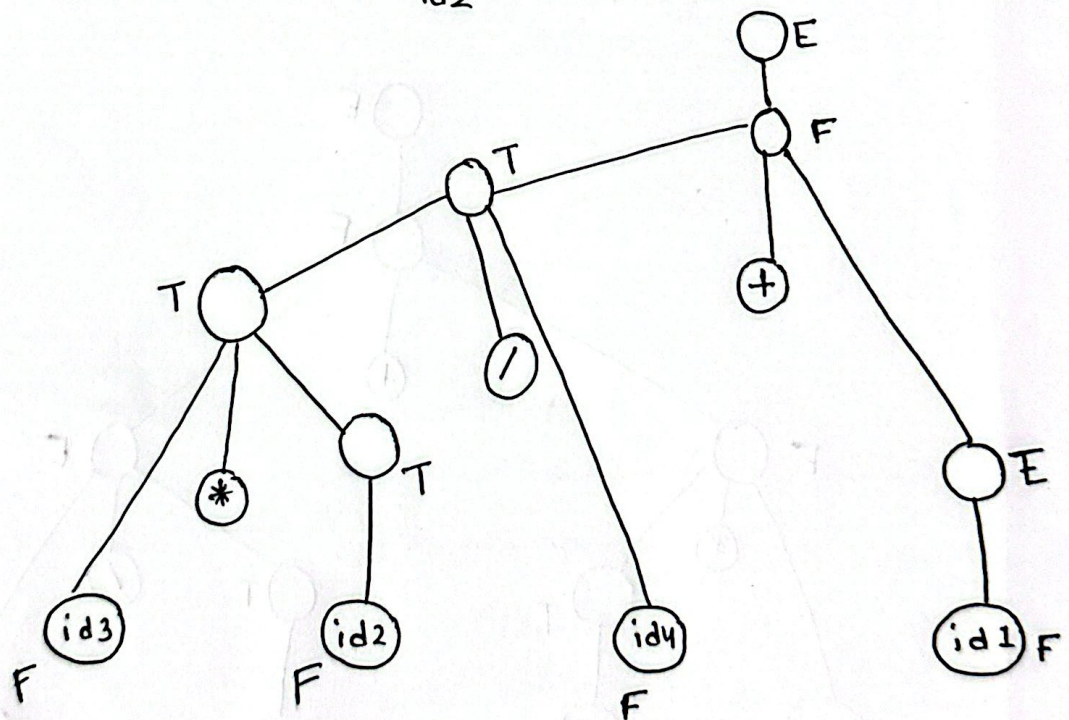
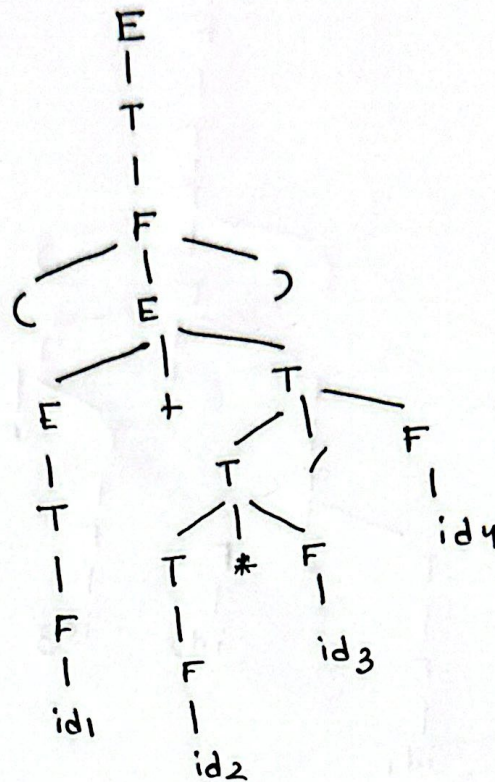


CO4	4. Draw the parse tree of nodes – for the input sentence (id + id * id / id) using the following attribute grammar. Note that it is important that the ordering of nodes matches the SDD in your drawing. $E \rightarrow E_1 + T$ {E.node = new Node(); childList = new List(); childList.add(T.node); childList.add(new Leaf('+')); childList.add(E₁.node); E.node.children = childList;} $E \rightarrow T$ {E.node = T.node;} $T \rightarrow T_1 * F$ {T.node = new Node(); childList = new List(); childList.add(F.node); childList.add(new Leaf('*')); childList.add(T₁.node); T.node.children = childList;} $T \rightarrow T_1 / F$ {T.node = new Node(); childList = new List(); childList.add(T₁.node); childList.add(new Leaf('/')); childList.add(F.node); T.node.children = childList;} $T \rightarrow F$ {T.node = new Node(); childList = new List(); childList.add(F.node); T.node.children = childList;} $F \rightarrow id$ {F.node = new Leaf(id.lexeme);} $F \rightarrow (E)$ {F.node = E.node;}	10
-----	---	----

Summer 2025(A)-4

Input string: $(id_1 + id_2 * id_3 / id_4)$



4. Draw the parse tree of nodes – for the input sentence (id / id + id * id) using the following attribute grammar. Note that it is important that the ordering of nodes matches the SDD in your drawing.

$E \rightarrow E_1 + T$ {E.node = new Node(); childList = new List();
childList.add(T.node); childList.add(new Leaf('+'));
childList.add(E₁.node); E.node.children = childList;}

$E \rightarrow T$ {E.node = T.node;}

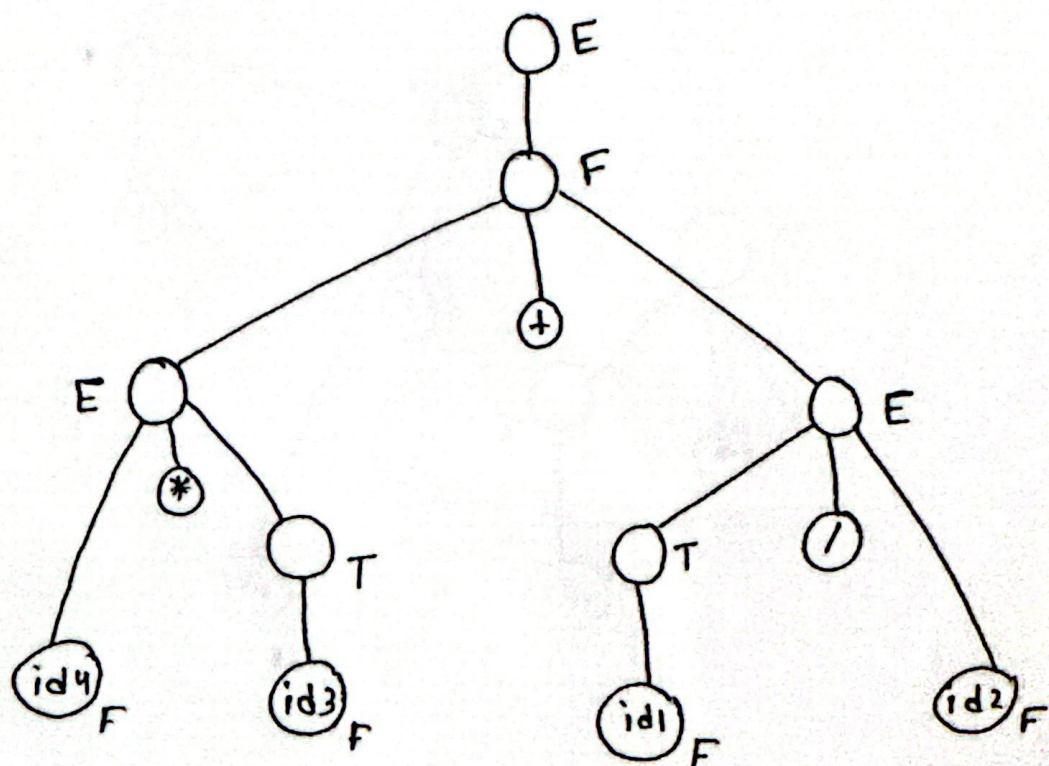
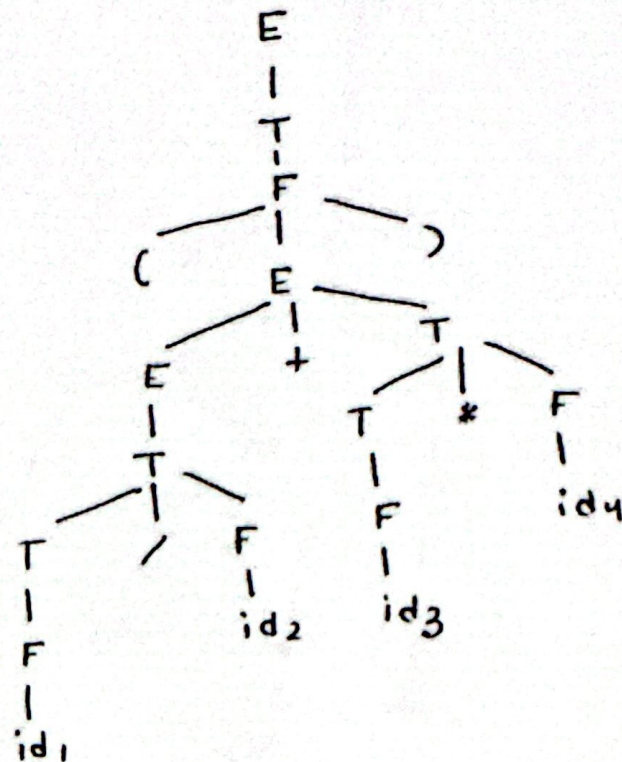
$T \rightarrow T_1 * F$ {T.node = new Node(); childList = new List();
childList.add(F.node); childList.add(new Leaf('*'));
childList.add(T₁.node); T.node.children = childList;}

$T \rightarrow T_1 / F$ {T.node = new Node(); childList = new List();
childList.add(T₁.node); childList.add(new Leaf('/'));
childList.add(F.node); T.node.children = childList;}

$T \rightarrow F$ {T.node = new Node(); childList = new List();
childList.add(F.node); T.node.children = childList;}

$F \rightarrow \text{id}$ {F.node = new Leaf(id.lexeme);}

$F \rightarrow (E)$ {F.node = E.node;}

Summer 2025(B) - 4Input string: $(id_1/id_2 + id_3 * id_4)$ 

6. You are provided with an abstract syntax tree and a set of productions of a grammar – for the
CO5 : input sentence $x + (y + z) * w$. You have to fill in the missing rules so that your semantic
rules match the ordering of the nodes in the parse tree.

5

$E \rightarrow E_1 + T$ {Write your rules here}

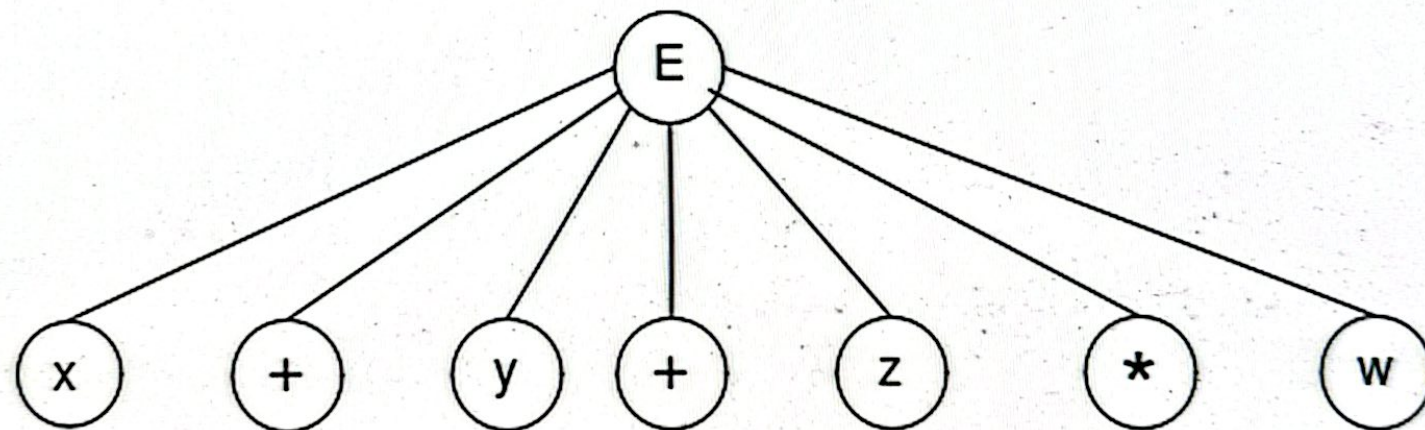
$E \rightarrow T$ {E.node = T.node;}

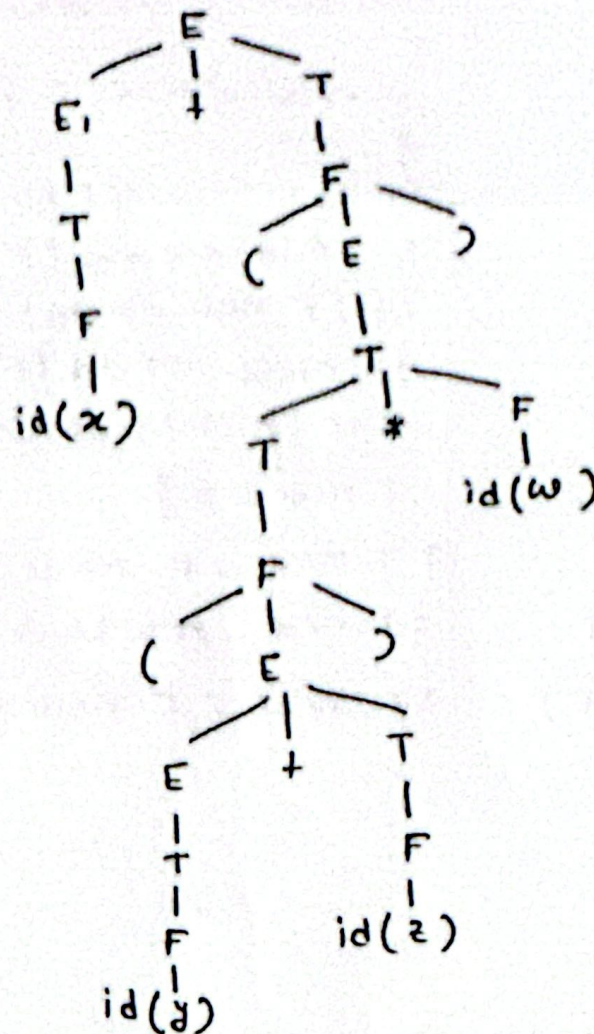
$T \rightarrow T_1 * F$ {Write your rules here}

$T \rightarrow F$ {T.node = F.node}

$F \rightarrow id$ {Write your rules here}

$F \rightarrow (E)$ {F.node = E.node}



Fall 2025 (A) - 6Input string: $x + ((y + z) * w)$ 

```

E → E1 + T { n = newNode(E);
                if (E1.node.child != empty)
                {
                    n.child.append(E1.node.child);
                }
                else { n.child.append(E1.node); }
                n.child.append("+");
                if (T.node.child != empty)
                {
                    n.child.append(T.node.child);
                }
                else { n.child.append(T.node); }
                E.node = n }
  
```

$E \rightarrow T$ $\{ E.\text{node} = T.\text{node}; \}$

$T \rightarrow T_1 * F$ $\{ n = \text{new Node}();$
 $\text{if } (T_1.\text{node}.\text{child} \neq \text{empty})$
 $\{$
 $n.\text{child}.\text{append}(T_1.\text{node}.\text{child});$
 $\}$
 $\text{else } \{ n.\text{child}.\text{append}(T_1.\text{node}) \}$
 $n.\text{child}.\text{append}("*");$
 $\text{if } (F.\text{node}.\text{child} \neq \text{empty})$
 $\{ n.\text{child}.\text{append}(F.\text{node}.\text{child}); \}$
 $\text{else } \{ n.\text{child}.\text{append}(F.\text{node}) \}$
 $T.\text{node} = n \}$

$T \rightarrow F$ $\{ T.\text{node} = F.\text{node} \}$

$F \rightarrow \text{id}$ $\{ F.\text{node} = \text{new Leaf}(\text{id}.\text{lexeme}) \}$

$F \rightarrow (E)$ $\{ F.\text{node} = E.\text{node} \}$

6. You are provided with an abstract syntax tree and a set of productions of a grammar – for the
CO5 input sentence $x + (y * z) + w$. You have to fill in the missing rules so that your semantic
rules match the ordering of the nodes in the parse tree.

5

$E \rightarrow E_1 + T$ {Write your rules here}

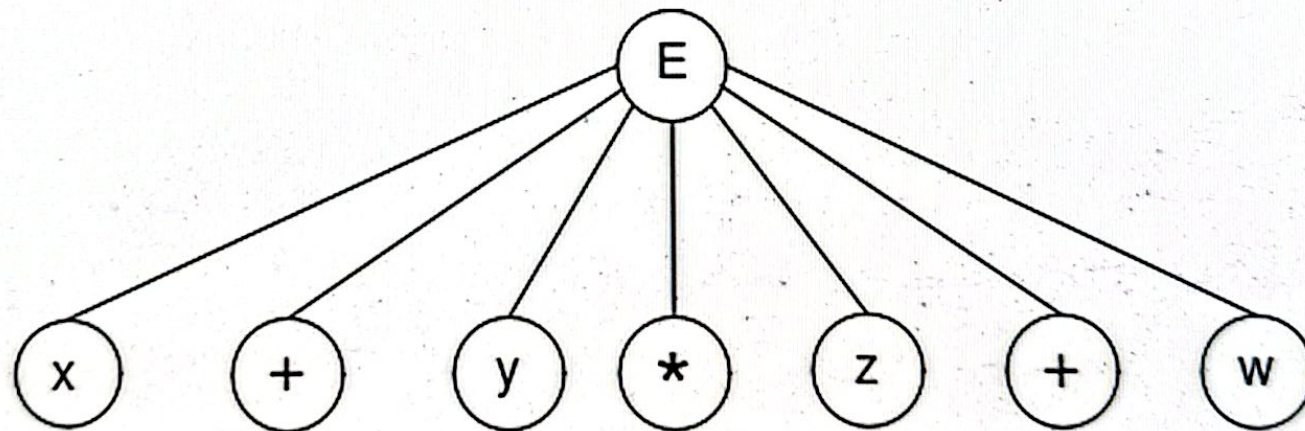
$E \rightarrow T$ {E.node = T.node;}

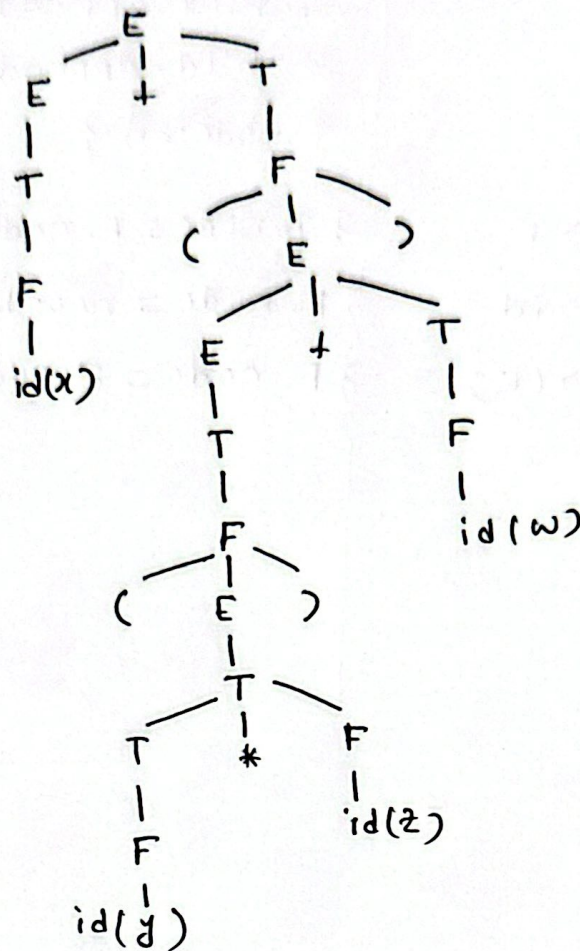
$T \rightarrow T_1 * F$ {Write your rules here}

$T \rightarrow F$ {T.node = F.node}

$F \rightarrow id$ {Write your rules here}

$F \rightarrow (E)$ {F.node = E.node}



Fall 2025(B)-6Input string: $x + ((y * z) + w)$ 

$E \rightarrow E_1 + T$ $\{ n = \text{newNode}(E);$
 $\text{if } (E_1.\text{node}.\text{child} \neq \text{empty})$
 $\{ n.\text{child}.\text{append}(E_1.\text{node}.\text{child}); \}$
 $\text{else } \{ n.\text{child}.\text{append}(E_1.\text{node}); \}$
 $n.\text{child}.\text{append}("+");$
 $\text{if } (T.\text{node}.\text{child} \neq \text{empty})$
 $\{ n.\text{child}.\text{append}(T.\text{node}.\text{child}); \}$
 $\text{else } \{ n.\text{child}.\text{append}(T.\text{node}); \}$
 $E.\text{node} = n \}$

$E \rightarrow T$ $\{ E.\text{node} = T.\text{node}; \}$

$T \rightarrow T_1 * F$

$T \rightarrow T_1 * F$ { $n = \text{newNode}()$;
 $n.\text{child}.\text{append}(T_1.\text{node})$;
 $n.\text{child}.\text{append}("*")$;
 $n.\text{child}.\text{append}(F.\text{node})$;
 $T.\text{node} = n$ }

$T \rightarrow F$ { $T.\text{node} = F.\text{node}$ }

$F \rightarrow \text{id}$ { $F.\text{node} = \text{new Leaf}(\text{id}.\text{lexeme})$ }

$F \rightarrow (E)$ { $F.\text{node} = E.\text{node}$ }

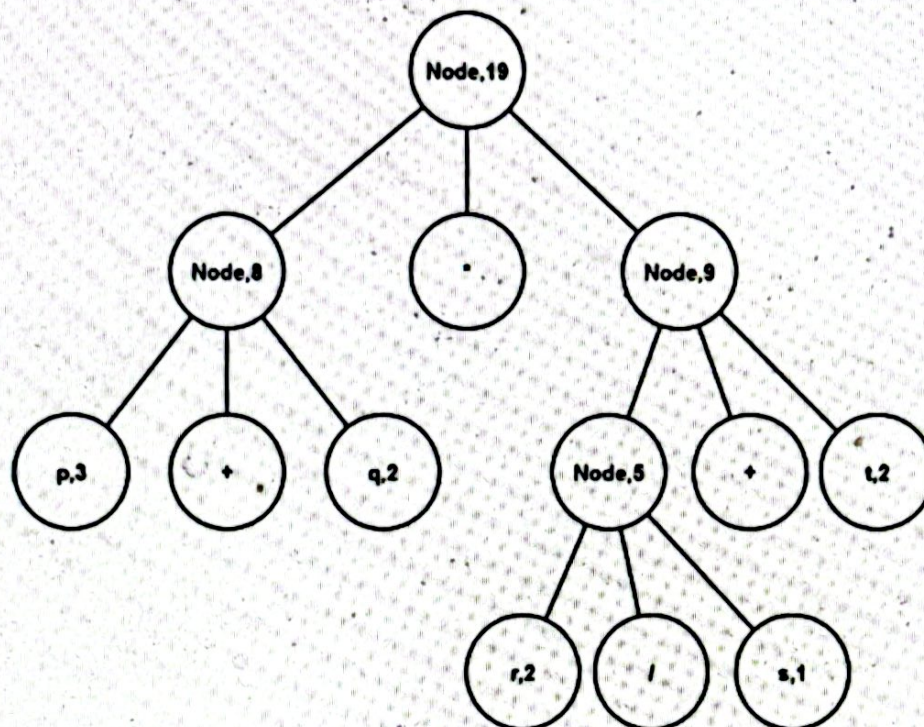
6.
CO4

Write the SDD rules for an abstract syntax tree construction for the following grammar that will store a count for the number of reduce operations being done to reach up to the current node for each node in the parse tree.

5

Input String: $(p+q)*(r/s+t)$

$E \rightarrow E_1 + T$ { Write your rules here }
 $E \rightarrow T$ { Write your rules here }
 $T \rightarrow T_1 * F$ { Write your rules here }
 $T \rightarrow T_1 / F$ { Write your rules here }
 $T \rightarrow F$ { Write your rules here }
 $F \rightarrow id$ { Write your rules here }
 $F \rightarrow (E)$ { Write your rules here }



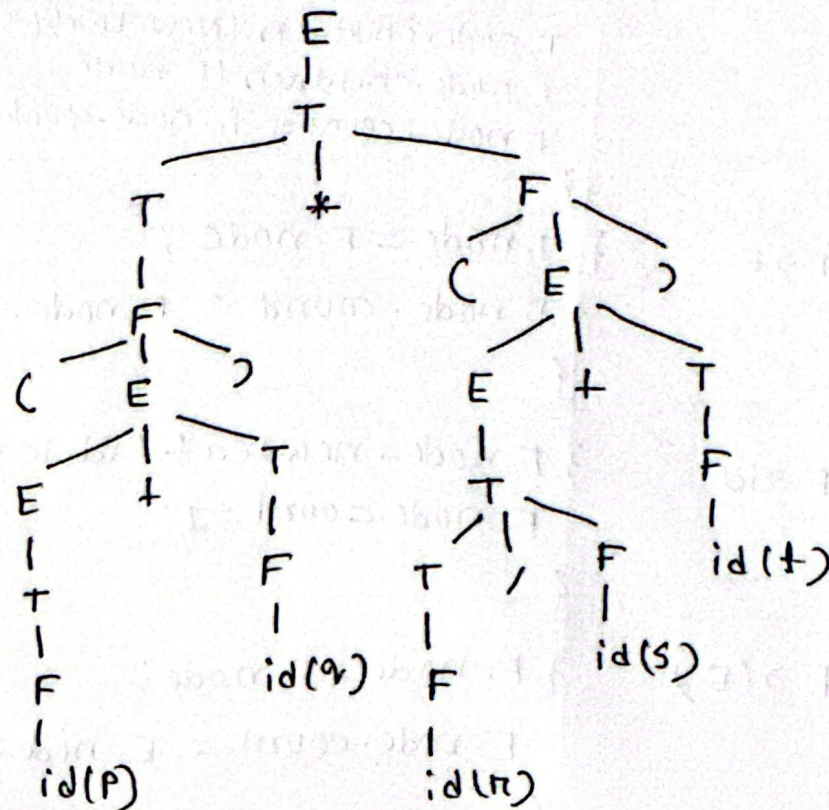
Hints:

For p,

the reductions are $F \rightarrow id$, $T \rightarrow F$, $E \rightarrow T$,
hence count = 3.

For the division node in r/s,

r undergoes $F \rightarrow id$ and $T \rightarrow F$ giving 2 reductions,
s undergoes $F \rightarrow id$ giving 1 reduction,
then $T \rightarrow T/F$ adds 1 reduction,
and $E \rightarrow T$ adds 1 reduction,
hence count = $2 + 1 + 1 + 1 = 5$.

Spring 2026 (A) - 6Input string : $(P+q) * (\pi/s+t)$ 

$E \rightarrow E_1 + T$ { $E.\text{node} = \text{new Node}();$
 $E.\text{node}.\text{children}(E_1.\text{node});$
 $E.\text{node}.\text{children}(\text{new Leaf}('+'));$
 $E.\text{node}.\text{children}(T.\text{node});$
 $E.\text{node}.\text{count} = E_1.\text{node}.\text{count} + T.\text{node}.\text{count} + 1; \}$

$E \rightarrow T$ { $E.\text{node} = T.\text{node};$
 $E.\text{node}.\text{count} = T.\text{node}.\text{count} + 1;$
 $\}$

$T \rightarrow T_1 * F$ { $T.\text{node} = \text{new Node}();$
 $T.\text{node}.\text{children}(T_1.\text{node});$
 $T.\text{node}.\text{children}(\text{new Leaf}('*'));$
 $T.\text{node}.\text{children}(F.\text{node});$
 $T.\text{node}.\text{count} = T_1.\text{node}.\text{count} + F.\text{node}.\text{count} + 1;$
 $\}$

$T \rightarrow T_1 / F$ { $T.\text{node} = \text{new Node}()$;
 $T.\text{node}.\text{children}(T_1.\text{node});$
 $T.\text{node}.\text{children}(\text{New Leaf}("/"));$
 $T.\text{node}.\text{children}(F.\text{node});$
 $T.\text{node}.\text{count} = T_1.\text{node}.\text{count} + F.\text{node}.\text{count} + 1;$
 }

$T \rightarrow F$ { $T.\text{node} = F.\text{node};$
 $T.\text{node}.\text{count} = F.\text{node}.\text{count} + 1;$
 }

$F \rightarrow \text{id}$ { $F.\text{node} = \text{new Leaf}(\text{id}.\text{lexeme});$
 $F.\text{node}.\text{count} = 1;$
 }

$F \rightarrow (E)$ { $F.\text{node} = E.\text{node};$
 $F.\text{node}.\text{count} = E.\text{node}.\text{count} + 1;$
 }